Hazard Analysis
Natural Control

Group 7
Lucas DeBoer
Kevin Kim
Aaron Loh
Curtis Vos

Table 1: Revision History

Date	**Developer(s)**	**Change**
2026-10-19	All	Initial version from team discussion

Contents

1 Introduction **1**

2 Scope and Purpose of Hazard Analysis **1**

3 System Boundaries and Components **1**

4 Critical Assumptions **2**

5 Failure Mode and Effect Analysis **3**

6 Safety and Security Requirements **8**
6.1 Functional Safety Requirements 8
6.2 Non-Functional Safety Requirements 10

7 Roadmap **10**

8 Appendix: Generative AI Use Disclosure **11**
8.1 AI Tools Used 11
8.2 How and Where Used 11
8.3 Verification of AI-Generated Content 11

9 Appendix: Reflection **12**

List of Tables

1 Revision History i
2 Failure Mode and Effect Analysis 4

1 Introduction

The purpose of this project is to create a voice-based interface for a computer-use agent that aims to help visually impaired users operate their computer while receiving accessible feedback and retaining control. For the purpose of this project, a hazard is defined as any property, condition, or event that could lead to harm or loss. Physical harm is not expected, so harm is defined as a loss of the user's intended outcome, control, awareness, privacy, or account security.

2 Scope and Purpose of Hazard Analysis

The purpose of this document is to identify and evaluate the hazards that may arise from the use of the system by considering the potential failure modes and their effects and causes, and then mitigating these risks through relevant software requirements. The hazards below could lead to these losses:

- **Unwanted action:** Something could be sent, submitted, purchased, created, or deleted that the user did not intend. This may be costly, impossible to undo, or affect other people.
- **Missed or failed task:** An action that the user believes happened does not happen, or happens incorrectly.
- **Loss of control or awareness:** The user cannot tell what the agent is doing, cannot stop it, or is told something false.
- **Privacy leak:** Private information or credentials could be exposed.
- **Loss of independence:** If the tool slows down, stalls, or stops, the user may no longer be able to operate their computer because they may need sighted help.

These losses are amplified as a sighted user can glance at the screen to catch a mistake; a visually impaired user often cannot, so detection has to be built into the system.

3 System Boundaries and Components

The list below outlines the major components of the system and their roles. This is a high-level view of the system that is used to brainstorm potential hazards and is used to complete the Failure Mode and Effects Analysis (FMEA) in Section 5.

1. **Accessible client:** The layer that supports app functions outside voice control and task execution, such as start and stop keyboard shortcuts and viewing task history.

2. **Voice interface:** Captures and transcribes user voice input, and provides text-to-speech audio feedback.

3. **Task manager:** Starts, stops, and tracks tasks.

4. **AI agent:** Plans tasks and calls tools.

5. **Action gate:** Handles approval of consequential actions.

6. **Outcome verification:** Verifies completed actions match what the user asked for.

7. **Action tools and integrations:** Tools the agent uses to gather information and perform actions.

8. **Task history layer:** Stores pending approvals, action history, task state, and any other relevant information needed to support recovery after disconnects.

The following are outside the system boundary: the user, laptop hardware (including the microphone and speakers), the operating system (macOS), the network, AI and speech service providers, websites, and external accounts and services. Failures of these are not analyzed internally, but the system's response to them is.

4 Critical Assumptions

1. Setup done correctly.
   A setup helper completed the assisted setup (accounts connected, permissions granted), so setup errors are not analyzed as runtime hazards. Setup help may only be available weekly (for example, from an intervenor), so recovery after setup should not depend on it.

2. One authorized speaker.
   There is only one user giving commands; other voices are noise. We do not need to verify who is speaking.

3. User's own accounts.
   Connected accounts belong to the user, who may act on them. Development and evaluation use test accounts.

4. Good faith user.
   The user does not purposely misuse the system or cause harm or loss. The user also does not intentionally give misleading or vague commands; they always strive to give instructions to the best of their ability.

5. Compatible device.
   The user has a functioning laptop with a microphone and speakers running macOS with Chrome installed. Note that we will consider scenarios where

the laptop has a microphone and speakers that fail mid-usage, but all laptops are assumed to have these components.

6. Users can understand spoken output.
   We assume that the user can understand the spoken output of the system. We do not analyze scenarios where the user has both vision and hearing loss. This excludes many deafblind users, such as the clients of CNIB Deafblind Community Services (from our October 17 interview); braille output through VoiceOver is a possible future extension.

We do not assume that the network, speech recognition, the AI model, websites, or other external services work correctly. Their failures are analyzed in the FMEA.

5 Failure Mode and Effect Analysis

The following table lists potential failure modes of each component, their effects and causes, recommended actions to mitigate the hazards, and the associated safety requirements. Each row is one failure mode of one component. Hazard IDs are `H<component>-<letter>`. How a failure is detected without sight is part of the recommended action, since a blind user often cannot notice a failure visually.

Table 2: Failure Mode and Effect Analysis

Ref	Component	Failure mode	Effects	Causes	Recommended action	SR
H1-a	Accessible client	Stop control can't be reached without voice	User can't halt the agent if voice fails; loss of control	Shortcut clashes with VoiceOver; focus in another app; client frozen	Provide a stop shortcut that works whenever the client runs and doesn't clash with VoiceOver; test with VoiceOver on	FR.Sa.1
H2-a	Voice interface	Misunderstands the user: wrong words, names, numbers, a cut-off sentence, or another voice	Wrong task or wrong details acted on	Background noise, accents, turn detection ending too early, TV or other people	Read back key details before every consequential action and require explicit approval before continuing. Exception outlined in H5-a for optional cookies.	FR.Sa.3
H2-b	Voice interface	A stop request isn't recognized	Task keeps running; loss of control	Stop request classified as chatter; mistranscription; user talking over the system	Recognize stop phrases with a fixed rule before any model interprets them; keyboard stop as backup; acknowledge quickly	FR.Sa.4, NFR.Sa.1
H2-c	Voice interface	Cannot receive or transcribe voice input or give voice output	User can't give commands, approvals, or stop, or can't hear feedback while the agent is working; loss of control and awareness	Transcription or speech service failure; input or output device disconnected or removed	Detect loss of spoken input or output where possible, pause the current task, and block new task actions until voice communication is restored and the user explicitly resumes	FR.Sa.20, FR.Sa.5.i
H3-a	Task manager	Stop is acknowledged but new work starts	Unwanted actions after stop; false sense of control	Interruption occurs only at tool boundaries; an operation is already submitted	Treat "stop working" as pause. Block new task actions and preserve context. Report acknowledgment and confirmed pause separately. Explain what completed and what remains uncertain. Resume only on a clear user request. "Cancel" ends the task. Earlier actions are not undone.	FR.Sa.5.i, FR.Sa.5.ii, FR.Sa.5.iii

Ref	Component	Failure mode	Effects	Causes	Recommended action	SR
H3-b	Task manager	An action is repeated after reconnecting	Duplicate task	Automatic retry after a dropped connection when the first attempt actually succeeded	Unique ID per approved action; never retry an action automatically; pause new actions while disconnected; on reconnect, require explicit user approval before retrying	FR.Sa.6, FR.Sa.7, NFR.Sa.2
H4-a	AI agent	Does the wrong task or picks the wrong target	Wrong action prepared (wrong thread, wrong date)	Ambiguous request; model error	Ask a clarifying question when required details are missing or ambiguous; the approval read-back (H2-a) catches the rest	FR.Sa.9
H4-b	AI agent	Claims outcome that didn't happen	Missed task; duplicated task; user misinformed	Model hallucination; partial failure reported as success; partial success reported as failure	Report completion only from verified outcomes; distinguish verified, partial, failed, and unknown results	FR.Sa.10
H4-c	AI agent	Stalls or loops	Silence or repeated steps; user unsure what's happening; cost grows	Model loop; slow or broken website	Provide meaningful spoken progress or blocker updates before a configurable maximum silence interval is exceeded during active task execution; pause and ask the user after a step or execution-time limit.	FR.Sa.11, NFR.Sa.3
H4-d	AI agent	AI provider outage or timeout	Task halts partway	Provider down; network loss	Pause and tell the user what was and wasn't done; never claim success	FR.Sa.12
H5-a	Action gate	A consequential action slips through without approval	Unwanted action with lasting effects runs (delete, purchase, submit)	Action not recognized as consequential (e.g. a browser click that submits a form); new tool not classified	Require explicit approval for consequential actions and actions with unclear effects. One exception: automatic rejection of non-essential cookies on webpages.	FR.Sa.13
H5-b	Action gate	A stale approval is reused	Action runs with details the user never approved	Details changed after approval (time, recipient)	Bind each approval to its exact details; approvals are single-use; any change requires new approval	FR.Sa.14

Ref	Component	Failure mode	Effects	Causes	Recommended action	SR
H5-c	Action gate	A pending approval expires or is withdrawn without a clear result	User unsure whether the action happened	Pause, disconnection, cancellation, or task replacement	When a pending approval expires or is withdrawn, the system shall tell the user that the action was not executed and shall state which earlier actions were completed.	FR.Sa.15
H5-d	Action gate	A delete is approved by mistake	Item or data lost, potentially permanently	Misheard approval; wrong item matched; user assumes the action can be undone	Before deletions, read back identifying details, explain whether the operation is reversible, and prefer recoverable deletion (trash) where supported.	FR.Sa.19
H6-a	Outcome verification	Verification wrongly confirms success, or cannot check and reports success anyway	User believes a task succeeded when it did not; missed task	Checking the wrong item or a stale copy of the page or account; the service does not expose the result	Re-read the specific item the action changed after acting; if it cannot be checked, report the result as unknown	FR.Sa.10
H6-b	Outcome verification	Verification wrongly reports failure, or cannot check and reports failure anyway	User retries, causing a duplicate action; user loses trust in verification	Checking a stale copy or the wrong item; the service updates slowly after the action	Before reporting failure re-check the item; if the result is still uncertain, report it as unknown, not failed	FR.Sa.10
H7-a	Action tools and integrations	Acts on the wrong page element	Wrong item chosen or wrong field filled	Poor labels; unusual layout; page changed after reading	Re-check the target against the approved details immediately before acting; verify the result afterward	FR.Sa.10, FR.Sa.14
H7-b	Action tools and integrations	Follows instructions injected by a webpage	Unwanted action or private data sent elsewhere	Page or email text written to manipulate agents	Treat page and email content as data, never as commands; consequential actions must trace to the user's request; tell the user when content tries to give instructions	FR.Sa.16
H7-c	Action tools and integrations	Access is broader than needed	Any other failure can do more damage	Requesting broad OAuth permissions for convenience	Request the narrowest permissions each tool needs; keep tokens out of the repository and logs; store and use passwords securely	FR.Sa.17

Ref	Component	Failure mode	Effects	Causes	Recommended action	SR
H7-d	Action tools and integrations	Outcome unknown after a timeout	Retrying risks a duplicate; not retrying risks a missed task	Network or API timeout	Check the service or item state directly and get user approval before any retry	FR.Sa.18
H8-a	Task history layer	Task history doesn't match what happened	User can't review or catch wrong actions afterward	History written from the agent's claims instead of recorded events; events lost during disconnects	Build history only from recorded actions and verified outcomes; label anything unknown	FR.Sa.2
H8-b	Task history layer	Task state or pending approvals lost	User never asked, or told the wrong status	Crash or restart of the task manager or agent	Save task state and approval records; after a restart, announce interrupted tasks and their last known status; unused approvals shall be withdrawn	FR.Sa.8

6 Safety and Security Requirements

These requirements come from the recommended actions in Table 2 and are also part of the SRS. FR.Sa marks functional safety requirements and NFR.Sa marks non-functional ones. Timing values are named constants, set after POC measurements (see the constants table in section S.2.8 of the SRS).

6.1 Functional Safety Requirements

- **FR.Sa.1:** The system shall provide a keyboard stop control that works whenever the client is running and does not conflict with VoiceOver commands. (H1-a)
- **FR.Sa.2:** The system shall build the task history only from recorded actions and verified outcomes, labelling unknown entries. (H8-a)
- **FR.Sa.3:** Before a consequential action the system shall read back its key details. It shall proceed only after explicit approval. The cookie-rejection exception is defined in FR.Sa.13. (H2-a)
- **FR.Sa.4:** Explicit stop phrases and the stop key shall use a fixed control path. They shall not depend on the planning agent choosing to pause. (H2-b)
- **FR.Sa.5.i:** After a stop-work request, the system shall pause the task and block all further actions for it. Only checks of earlier outcomes may continue running. (H2-c, H3-a)
- **FR.Sa.5.ii:** After a stop-work request, the system shall immediately acknowledge and confirm completion of the request as separate messages before reporting completed work, unfinished work, and uncertain outcomes. (H3-a)
- **FR.Sa.5.iii:** When a task is paused, an explicit user request shall resume or terminate the task. If terminated, earlier actions are not undone. (H3-a)
- **FR.Sa.6:** Each approved action shall have a unique ID. An action with that ID shall not execute twice. (H3-b)
- **FR.Sa.7:** Disconnection shall block new task actions and pause the task. Reconnection alone shall not resume it. (H3-b)
- **FR.Sa.8:** Task state and approval records shall survive restart. The system shall announce interrupted tasks with their last known state. Unused approvals shall be withdrawn. (H8-b)
- **FR.Sa.9:** The system shall ask a clarifying question when details required for an action are missing or ambiguous. (H4-a)

- **FR.Sa.10:** The system shall report a task as verified, failed, or partial only after verifying its outcome with evidence that matches the intended action and its details, such as a connector result, an account record, or a website confirmation. If the evidence is unclear, the system shall check again or report the outcome as unknown. (H4-b, H6-a, H6-b, H7-a)
- **FR.Sa.11:** A configured step or time limit shall pause the task and prompt the user. Time spent waiting for user clarification or approval shall not count toward the execution time limit. (H4-c)
- **FR.Sa.12:** An external service failure shall pause work and the system shall follow the steps outlined in FR.Sa.5.i, FR.Sa.5.ii, and FR.Sa.5.iii. (H4-d)
- **FR.Sa.13:** Unclear actions and consequential actions such as submissions, external messages, saved data changes, or granting permission require explicit approval before being executed; only actions with no such lasting effect do not require approval. One exception to this rule: rejecting non-essential cookies or only allowing necessary cookies on webpages does not require explicit approval. (H5-a)
- **FR.Sa.14:** The system shall bind each approval to the action's exact details, use it at most once, and require new approval if any detail changes. A pause, disconnection, cancellation, or task replacement shall withdraw unused approvals. (H5-b, H7-a)
- **FR.Sa.15:** When a pending approval expires or is withdrawn, the system shall tell the user that the action was not executed and shall state which earlier actions were completed. (H5-c)
- **FR.Sa.16:** The system shall treat webpage and email content as data, and shall only perform consequential actions that trace to the user's request. (H7-b)
- **FR.Sa.17:** The system shall request only the account permissions each tool needs, and shall store and use passwords and other credentials securely, keeping them out of source control, logs, and the AI model input. (H7-c)
- **FR.Sa.18:** The system shall check the service or item state directly and confirm the user's approval before retrying an action whose outcome is unknown. (H7-d)
- **FR.Sa.19:** Before deletions, the system shall read back the item's identifying details, state whether the operation is reversible, and use recoverable deletion where supported. (H5-d)
- **FR.Sa.20:** If voice input, voice output, or speech services become unavailable, the system shall stop as outlined in FR.Sa.5.i, accept no new tasks requests until voice functionality is restored, and shall not resume unless user explicitly requests it. (H2-c)

6.2 Non-Functional Safety Requirements

- **NFR.Sa.1:** The system shall audibly acknowledge a stop request within STOP_ACK_TIME of the end of the utterance or key press. (H2-b)
- **NFR.Sa.2:** The client shall signal disconnection within CONNECTION_NOTICE_TIME. A local sound shall be used if network speech is unavailable. (H3-b)
- **NFR.Sa.3:** While a task is running, the system shall give a spoken status update after IDLE_PROMPT_TIME without spoken progress, outlining progress made or blockers. (H4-c)

7 Roadmap

POC (November). FR.Sa.3, 4, 5 (i, ii, iii), 13, 14; NFR.Sa.1. The safeguards the demo is meant to prove. FR.Sa.13 includes testing whether browser actions can be intercepted.

Before Rev 0 / participant evaluation. FR.Sa.1, 2, 6, 7, 8, 9, 10, 12, 15, 16, 17, 18, 19, 20; NFR.Sa.2. These matter as soon as real participants use real-looking accounts.

Later. FR.Sa.11; NFR.Sa.3. Basic status updates exist earlier, but tuned step/time limits and update frequency need usability evidence. (Team to confirm.)

8 Appendix: Generative AI Use Disclosure

8.1 AI Tools Used

Aaron Loh used Claude Code with Claude Opus 5.5. Kevin Kim used OpenAI Codex with GPT-6.1 Sol. Lucas DeBoer used Claude Sonnet 5.5 through the claude.ai web interface.

8.2 How and Where Used

Aaron's uage was representative of the whole team. The team had a meeting where they talked through the answers to all the sections of the Hazard Analysis. Using the transcript of the meeting and the rubric, Claude asked them followup questions to clarify scope, losses, components, and failure modes. Then, using the transcript and the responses to the follow up questions, Claude generated an initial draft of the Hazard Analysis. Kevin Kim used Codex to help improve the clarity of H3-a and FR.Sa.5 and make them more cohesive with the rest of the document. Lucas DeBoer used Claude for grammatical assistance throughout the document and to generate feedback on the FMEA table and the safety requirements.

8.3 Verification of AI-Generated Content

Kevin used AI for a very small portion, so he fully read through the suggested changes and verified their accuracy and appropriateness before accepting them. As Aaron's usage was to create a rough draft of the entire document, and the content was based on the transcript of the team meeting, the team read over the draft and verified that the content matched their discussion. Then, when Lucas was going through the document creating the final version, he read through everything in-depth and modified the content to ensure it was accurate and correct. Lucas verified all grammatical suggestions from Claude before accepting them. He also did not use Claude to generate any content in this document. Rather he used Claude as an extra set of eyes to help generate feedback on his work. Lucas fully reviewed all feedback and only implemented suggestions if he deemed them applicable.

9 Appendix: Reflection

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

Kevin Kim

1. **What went well while writing this deliverable?**

   One thing that went well was using the Hazard Analysis as a foundation for the safety requirements in our SRS. Although I wasn't directly involved in writing the initial Hazard Analysis, I reviewed both documents to identify inconsistencies and ensure the requirements aligned. This helped us clarify important safety behaviors involving task interruptions, user approvals, and error handling.

2. **What pain points did you experience, and how did you resolve them?**

   The main challenge was resolving inconsistencies between the Hazard Analysis and SRS while keeping our requirements realistic for the project. I compared conflicting requirements and helped determine which approaches were more feasible. We also had to prioritize the safety features needed for our Proof of Concept rather than attempting to implement everything immediately.

3. **Which risks were identified before and during the Hazard Analysis?**

   Before the Hazard Analysis, our team had already considered risks such as misunderstood voice commands, missing user approvals, and insufficient spoken feedback. During the analysis, we examined more specific failure scenarios, such as duplicate actions after disconnection, incorrect completion reports, and tasks continuing after a stop request. These risks became clearer by examining how individual system components could fail and how those failures might affect users.

4. **What are two important types of software risk besides physical harm?**

Two important risks are privacy and security issues, and incorrect system actions. Natural Control may access sensitive emails, calendars, and account information, so protecting credentials and limiting permissions is important. The system could also perform an unintended action or incorrectly report that a task succeeded. Since users rely on spoken feedback, verifying outcomes and requiring approval for consequential actions are essential for maintaining user trust.

Aaron Loh

1. **What went well while writing this deliverable?**

   Working through the Hazard Analysis really helped me and the team in clarifying what we are actually building and being able to think about what could go wrong. I found it super helpful because it forced us to think through all of the actual use cases of how it might be used and go through the different situations users could end up in.

2. **What pain points did you experience and how did you resolve them?**

   One of the hardest things we faced was in defining our scope and really making sure that we kept it consistent across both our Hazard Analysis and the SRS. We really wanted to be ambitious with what we were doing but we also really needed to keep it scoped. We had quite a few discussions there figuring that out.

   Another part that was a little bit difficult was when we were defining what exactly stopping work means. We had to really think through the different situations where something might already be happening and what the system should do at that point. Going through those situations together helped us make things clearer.

3. **Which risks had you already thought of? Which came up during the analysis?**

   We had already thought about most of the general risks of an agent misunderstanding someone or doing something they didn't want it to do. Designing that in a way that's secure and safe was already something we knew we would have to learn a lot about.

   Working through the Hazard Analysis really forced us to think through some of the less obvious situations. For example we had to really think through what happens if the system says it stopped but then something is still happening in the background. Or what happens if an action worked but the system doesn't know that and tries it again? Thinking through the actual use cases helped us identify those risks.

4. **What are at least two risks other than physical harm? Why do they matter?**

   One would definitely be privacy and security. Our software is going to

interact with the external world and potentially have access to private information through connected accounts. Designing that in a way that's secure and safe is going to be really important.

Another would be doing something the user didn't actually want like sending the wrong message or changing something without their approval. Because they're mostly interacting through voice we really need to think through how they understand what's happening and stay in control.

There's also the risk of the system saying something worked when it didn't. That could leave someone thinking an important task was done when it wasn't so making sure the system reports things clearly is important.

Curtis Vos

1. **What went well while writing this deliverable?**

   The drafting process of the hazard analysis felt very productive, as all members were present in a long Discord call were we collaborated. After that drafting session, we split up the main review/finalization work between the HA/SRS documents, cross-reviewing each other's progress as we went on. The coordination and frequent updates that we maintained help make sure the work was completed as expected.

2. **What pain points did you experience, and how did you resolve them?**

   Maintaining consistency between this document and the SRS came to be a bit of a pain point for our group. If changes were made in the SRS, they had to be reflected in the hazard analysis, and vice versa, which caused a few problems when it came to splitting up the work among the group members. We had one such occasion where some wording of requirements got mixed up between the two, so I helped with an extra PR near the end to sync these documents up.

3. **Which risks were identified before and during the Hazard Analysis?**

   Some of the main risks that we thought of before beginning the deliverable were the system misunderstanding voice commands from the user and incorrectly claiming the results of outcomes without verifications. Working through the document presented a much larger number of risks that we didn't initially even have in mind, like the system stalling and looping, stale approvals being reused in new tasks, and following instructions that are mistakenly injected by webpages instead of the user's voice. These are more niche risks that only arise with deeper thought and exposure to our planning steps, proving the necessity of a dedicated hazard analysis.

4. **What are two important types of software risk besides physical harm?**

A major risk with a lot of software is privacy, as unsafe systems can expose users' private info, like banking details. Safely handling this to not be leaked or ever mishandled is vital so the user does not suffer personal hardships or financial losses, especially with systems now that deal with less-restricted AI models. Another risk with software products is performance. A slow-performing product may cause delays in completing important work for users at their jobs, may cause unnecessary stress, or may simply be inconvenient to use and cause the developers monetary losses from a poorly-performing product.